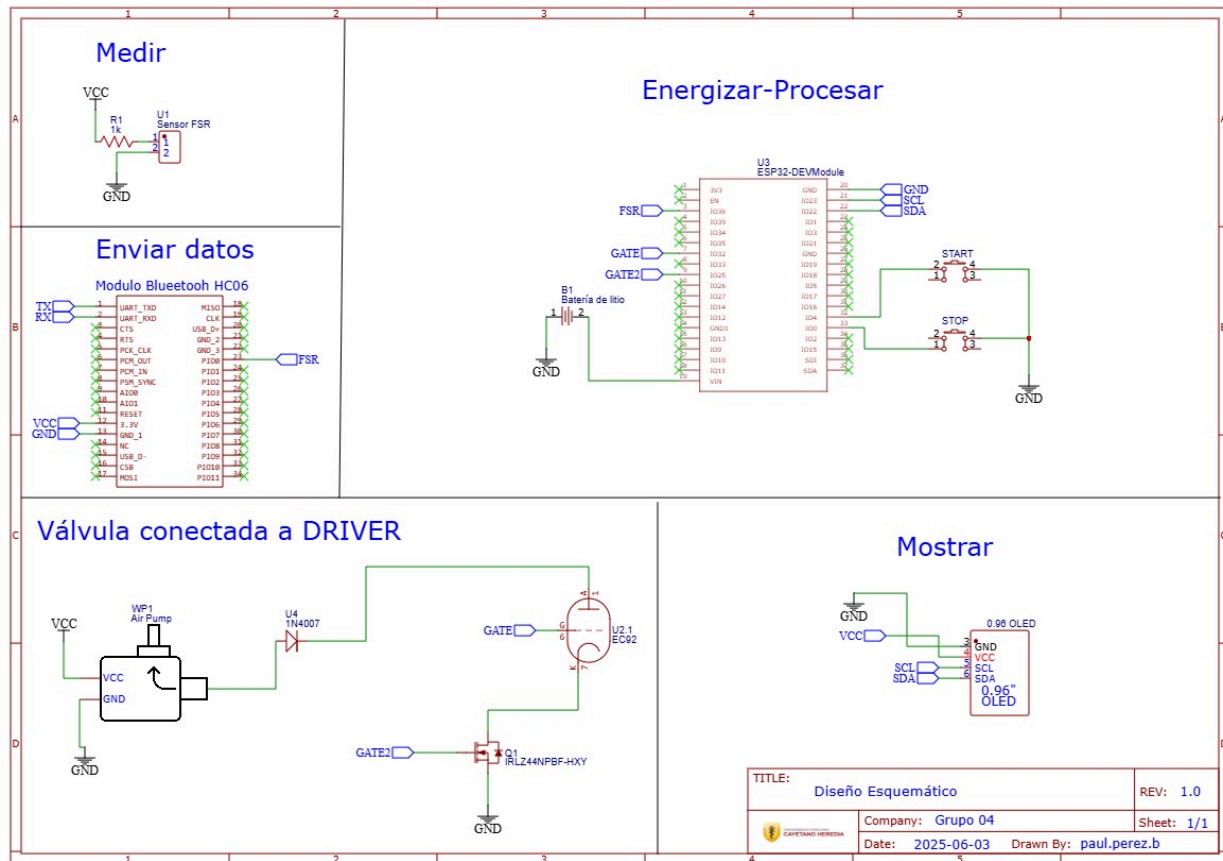


MODELADO 3D-AVANCE PROTOTIPADO ELECTRÓNICO

Hardware Electronico:

- Diseño esquemático del circuito electrónico (Versión final según observaciones del entregable anterior)



- Ejecución del código de esp32 con componentes físicos implementados.

Propuesta del código final:

Código:

```
#include <Arduino.h>
```

```
const int fsrPins[4][3] = {
  {34, 35, 32}, // Celda A
  {33, 25, 26}, // Celda B
  {27, 14, 12}, // Celda C
```

```

    {13, 23, 22} // Celda D
};

const int bombaPins[4] = {4, 16, 17, 5};

const int valvInflaPins[4] = {18, 19, 21, 3};

const int valvDesinflaPins[4] = {1, 2, 15, 0};

const int umbral_alto = 200;

const unsigned long tiempo_inflado = 4000;

bool enInflado[4] = {false, false, false, false};
unsigned long tiempoInicio[4] = {0, 0, 0, 0};

void setup() {
    Serial.begin(115200);
    for (int i = 0; i < 4; i++) {
        pinMode(bombaPins[i], OUTPUT);
        pinMode(valvInflaPins[i], OUTPUT);
        pinMode(valvDesinflaPins[i], OUTPUT);
        digitalWrite(bombaPins[i], LOW);
        digitalWrite(valvInflaPins[i], LOW);
        digitalWrite(valvDesinflaPins[i], HIGH);
    }
}

void loop() {
    for (int i = 0; i < 4; i++) {
        int maxPresion = 0;
        for (int j = 0; j < 3; j++) {
            int lectura = analogRead(fsrPins[i][j]);
            if (lectura > maxPresion) {

```

```

        maxPresion = lectura;
    }
}

if (!enInflado[i] && maxPresion >= umbral_alto) {

    Serial.print("Inflando celda "); Serial.println(i);
    digitalWrite(valvDesinflaPins[i], LOW);
    digitalWrite(valvInflaPins[i], HIGH);
    digitalWrite(bombaPins[i], HIGH);
    tiempoInicio[i] = millis();
    enInflado[i] = true;
}

if (enInflado[i] && millis() - tiempoInicio[i] >= tiempo_inflado) {

    Serial.print("Desinflando celda "); Serial.println(i);
    digitalWrite(bombaPins[i], LOW);
    digitalWrite(valvInflaPins[i], LOW);
    digitalWrite(valvDesinflaPins[i], HIGH);
    enInflado[i] = false;
}
}

delay(50);
}

```

Explicación del código:

Este programa implementa un sistema automático de regulación de presión en cuatro celdas neumáticas (A, B, C y D) utilizando sensores FSR402 conectados a un microcontrolador ESP32. Cada celda cuenta con su propia bomba de inflado y dos válvulas: una para inflar y otra para desinflar. El objetivo es detectar cuándo una celda experimenta una presión elevada, inflarla por un tiempo determinado y luego permitir su desinflado para evitar zonas de alta presión que puedan causar úlceras por presión.

Primero se definen los pines analógicos del ESP32 conectados a los sensores FSR, agrupados de tres en tres por celda. También se definen los pines digitales encargados de controlar individualmente las bombas, las válvulas de inflado y las de desinflado. En la función `setup()`,

estos pines se inicializan como salidas, se apagan las bombas y válvulas de inflado, y se abren las válvulas de desinflado por defecto, permitiendo que las celdas comiencen sin aire acumulado.

Durante la ejecución continua del programa (`loop()`), se realiza una lectura de los tres sensores de cada celda y se considera el valor más alto como representativo. Si la presión detectada supera un umbral establecido (por ejemplo, 200), el sistema inicia un ciclo de inflado: se cierra la válvula de desinflado, se abre la válvula de inflado y se enciende la bomba correspondiente. Este estado se mantiene durante un tiempo fijo (por ejemplo, 4 segundos).

Después del tiempo de inflado, el sistema apaga la bomba, cierra la válvula de inflado y vuelve a abrir la de desinflado, permitiendo que la celda libere presión. Este ciclo se repite constantemente para cada celda, de manera independiente, lo que permite una redistribución de la presión localizada.

Este enfoque ofrece un control automático, celda por celda, que responde únicamente a condiciones de presión elevadas. No se requieren entradas manuales ni comparaciones complejas entre zonas. El sistema está diseñado para ser simple pero efectivo, y puede adaptarse a integraciones más avanzadas como comunicación inalámbrica, visualización de datos o control remoto si se desea expandir su funcionalidad.

- Avance de su prototipado electrónico.

Código usado en el avance del prototipado electrónico (una sola celda):

```
// Pin del FSR y del relé
```

```
const int fsrPin = A0;
```

```
const int relePin = 13;
```

```
// Umbrales
```

```
const int UMBRAL_ACTIVAR = 200; // Si >=, inicia inflado
```

```
const int UMBRAL_REINICIO = 150; // Si <, permite próximo ciclo de inflado
```

```
// Duración del inflado en milisegundos
```

```
const unsigned long TIEMPO_INFLADO_MS = 4000UL;

// Indica si actualmente estamos en un ciclo de inflado

bool bombaActiva = false;

// Marca el instante en que se encendió la bomba

unsigned long tiempoInicio = 0;


void setup() {

    pinMode(fsrPin, INPUT);

    pinMode(relePin, OUTPUT);


    // Suponemos lógica normal de relé: HIGH activa, LOW desactiva

    digitalWrite(relePin, LOW); // Relé en reposo al inicio


    Serial.begin(9600);

}


void loop() {

    int lecturaFSR = analogRead(fsrPin);

    Serial.print("FSR: ");

    Serial.println(lecturaFSR);


    unsigned long ahora = millis();
```

```
// Si no estamos inflando y se supera umbral de activación
if (!bombaActiva && lecturaFSR >= UMBRAL_ACTIVAR) {

    // Iniciar inflado

    digitalWrite(relePin, HIGH); // Activa relé, enciende bomba

    bombaActiva = true;

    tiempoInicio = ahora;

    Serial.println("Inflando globo...");

}

// Si estamos inflando y ya cumplimos los 4 segundos
if (bombaActiva && (ahora - tiempoInicio >= TIEMPO_INFLADO_MS)) {

    // Apagar bomba

    digitalWrite(relePin, LOW);

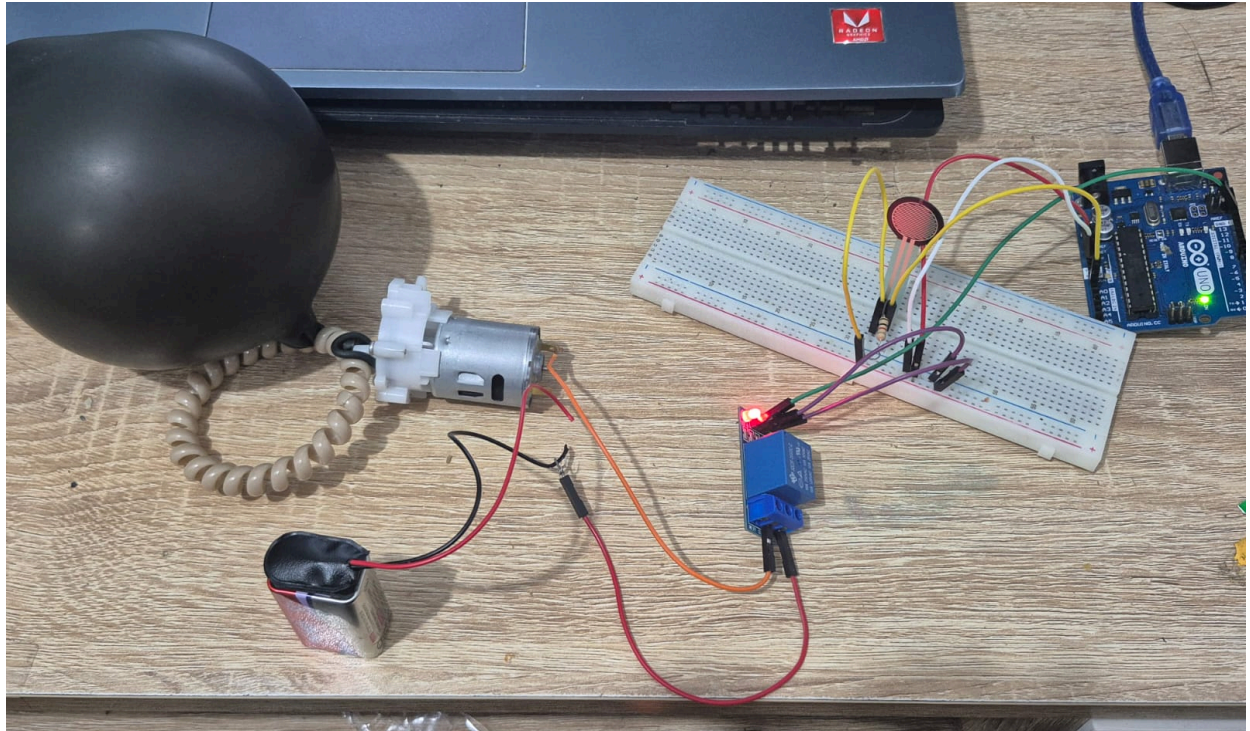
    bombaActiva = false;

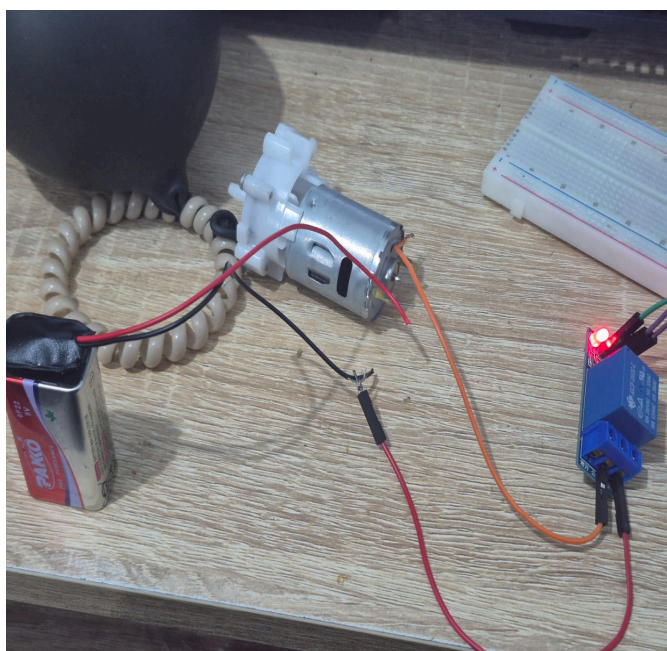
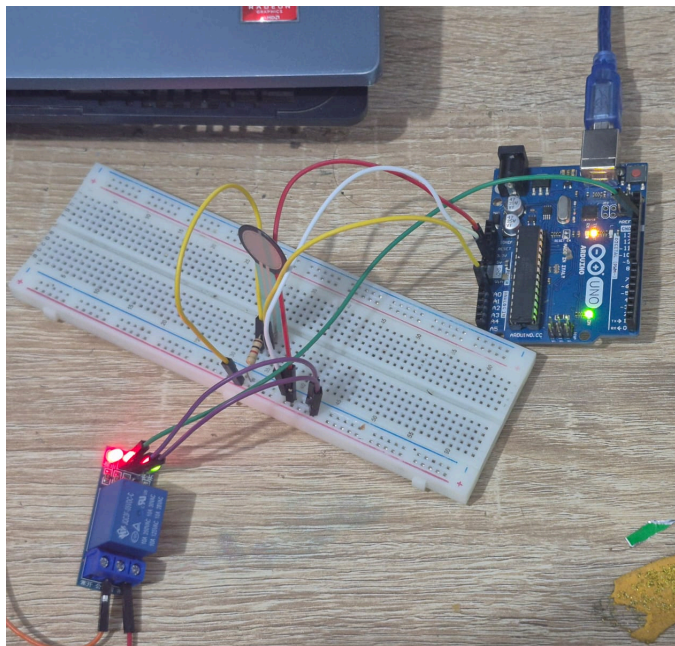
    Serial.println("Fin inflado. Esperando reinicio...");

}

delay(100); // Pequeña pausa antes de la siguiente lectura

}
```





En este primer avance del prototipo electrónico se está emulando cómo trabajaría la bomba de aire inflando la celda (simulada con un globo), con un solo sensor FSR que emite una señal al cruzar el umbral propuesto. Se utiliza un microcontrolador Arduino UNO en esta etapa, debido a que su entrada de 5 V es más tolerante frente al riesgo de sobrevoltaje que los 3.3 V del ESP32; esto se hace por precaución para no dañar el dispositivo en una fase inicial. Cuando el sensor detecta una lectura ≥ 200 , el Arduino envía una señal al módulo relé, el cual recibe la tensión de la batería externa (12 V) y la redirige a la bomba de aire, provocando el inflado del globo durante

un tiempo fijo (4 s). Transcurrido ese tiempo, el relé se desactiva y la bomba se apaga, permitiendo que el globo se desinfe de forma pasiva al liberarse la presión interna, en este avance hace falta el uso de válvulas de entrada y salida de aire, debido a que un sistema real de celdas neumáticas requiere controlar de modo preciso el ciclo de inflado y desinflado:

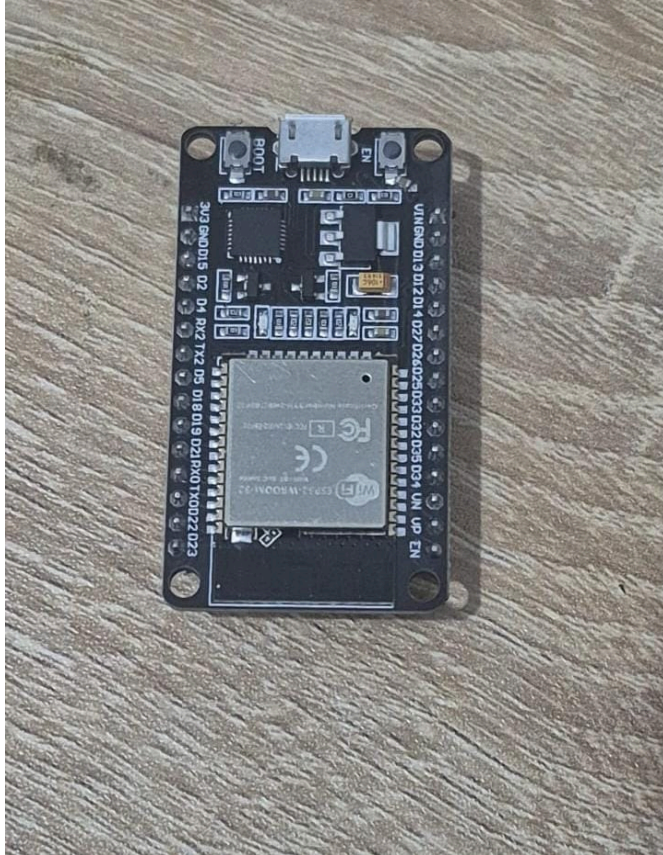
- Válvula de entrada: permitiría aislar la celda de la atmósfera durante el inflado, garantizando que la presión interna aumenta hasta el nivel deseado.
- Válvula de salida: al finalizar el inflado, debería abrirse para liberar aire controladamente y regresar la celda a un estado desinflado o de alivio de presión.

Sin esas válvulas, el globo se desinfla de modo pasivo e incontrolado, lo cual no reproduce fielmente la dinámica de Redistribución de Presión en una celda real. Además, la falta de aislamiento durante el inflado puede provocar lecturas erráticas o inflados continuos si el sensor FSR sigue detectando alta tensión sin un mecanismo que retenga el aire mientras se mide y ajusta la presión, en el informe de avance se debe indicar que, en la siguiente fase, se integrarán electroválvulas de entrada y salida controladas por el microcontrolador (ya sea Arduino o, en etapas posteriores, ESP32), el avance actual demuestra el principio de control de la bomba mediante sensor y relé, y en futuros prototipos se completará el ciclo real con válvulas dedicadas para entrada y salida de aire, mejorando la precisión, seguridad y reproducibilidad del sistema.

Válvula (entrada y salida del aire):

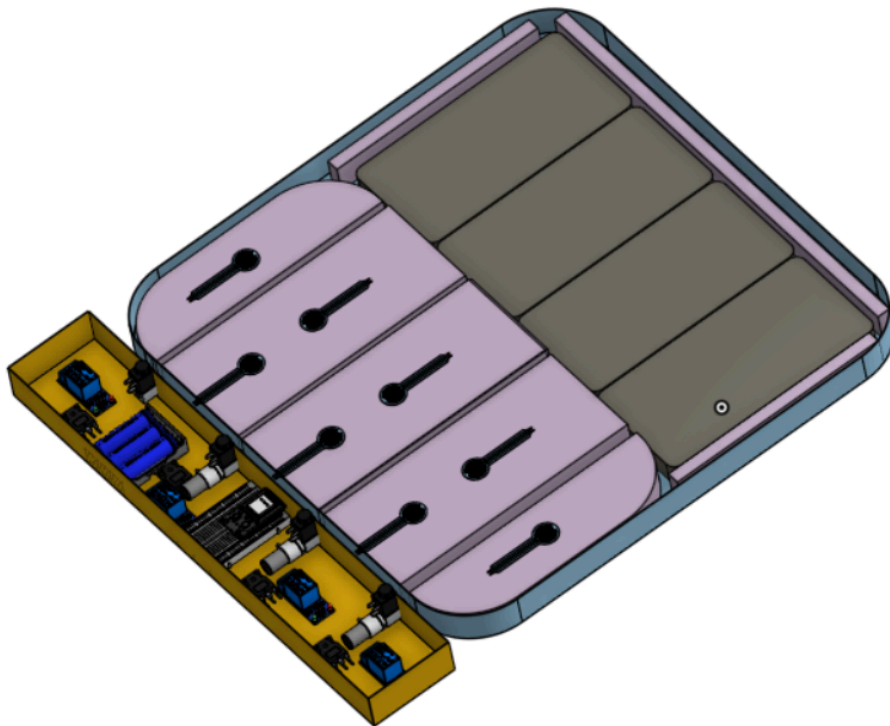
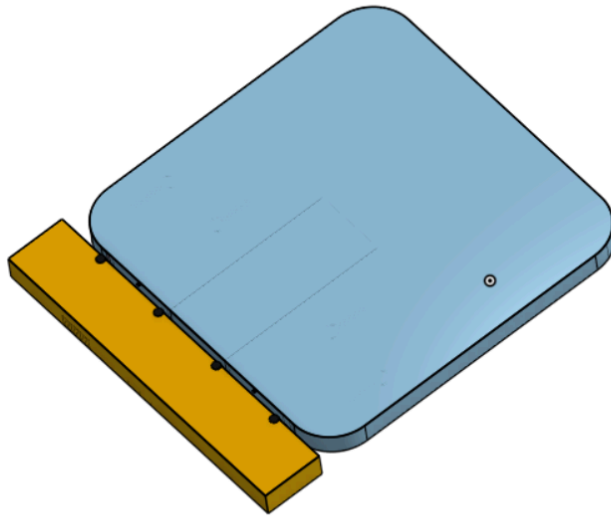


Microprocesador ESP32:



Fabricación digital:

- Modelado Digital de su prototipo completo, con todas los elementos modelados.



- Archivos DXF y planos de los modelos a corte laser

2. Perfil del usuario

El usuario objetivo de este cojín inteligente es una persona adulta mayor, rondando los 60 años, que presenta limitaciones físicas severas debido a una lesión medular cervical, como en el caso clínico considerado. Este usuario suele tener una movilidad reducida o nula en el tren inferior y necesita permanecer en posición sentada durante largos periodos, lo que lo expone al desarrollo de úlceras por presión. A nivel cognitivo, puede presentar desde un deterioro leve relacionado con la edad hasta capacidades conservadas, por lo que el diseño del dispositivo debe ser intuitivo, de bajo esfuerzo cognitivo y mínimamente intrusivo. En el aspecto emocional, estos usuarios valoran altamente la autonomía y la privacidad, por lo que es fundamental que el sistema funcione de manera automática y confiable, sin requerir intervención constante de cuidadores. Además, debido a la sensibilidad cutánea y al riesgo clínico, el cojín debe ser ergonómico, confortable y discreto, promoviendo no solo la prevención de lesiones sino también el bienestar emocional del usuario en su entorno cotidiano. Por lo tanto, consideramos que nuestro diseño puede prevenir posturas incorrectas en quienes no pueden realizar cambios y transferencias de peso de forma independiente y son propensos a deslizarse hacia adelante.

3. Análisis de tareas

Para garantizar el uso correcto del cojín inteligente de redistribución de presión, se han identificado una serie de tareas necesarias que el usuario o su entorno inmediato deben realizar. Estas incluyen: ajustar el cojín a la silla de ruedas, asegurar correctamente el cinturón de sujeción y asegurarse de tener bien las conexiones. Entre estas, se consideran tareas críticas aquellas que, si se ejecutan de forma incorrecta, podrían comprometer la funcionalidad del dispositivo o la seguridad del usuario. Por ejemplo, un mal ajuste del cojín podría derivar en una distribución inadecuada de la presión, aumentando el riesgo de úlceras por presión; o una mala conexión eléctrica podría impedir que el sistema funcione de manera autónoma, eliminando su capacidad de prevención activa. Estudios previos enfatizan que los adultos mayores, especialmente aquellos con deterioro cognitivo, requieren sistemas con interacciones simples y claras para evitar errores de uso (Carrigan *et al.*, 2019; Tavares *et al.*, 2022). Además, se ha demostrado que el diseño centrado en la facilidad de uso mejora considerablemente la aceptación del dispositivo en personas con movilidad reducida (Fadil *et al.*, 2022).

4. Criterios de éxito (requisitos de usabilidad)

Para determinar si el cojín inteligente desarrollado es usable y seguro para adultos mayores, se han definido una serie de criterios de éxito basados en estándares de usabilidad. En primer lugar, se evaluará la eficacia, observando si el usuario o cuidador puede completar correctamente tareas como la colocación del cojín y la activación del sistema sin asistencia técnica, con una tasa de éxito mínima del 90%. En cuanto a la eficiencia, se espera que el tiempo de instalación inicial del sistema no supere los 2 minutos, alineándose con métricas similares observadas en el estudio de Fadil *et al.* (2029). Para medir la satisfacción del usuario, se utilizará la escala SUS (System

Usability Scale), estableciendo como objetivo un puntaje mayor a 70, tal como fue reportado exitosamente en los estudios de Tavares et al. (2019) y Carrigan et al. (2022). Finalmente, en términos de seguridad, se buscará que no se produzcan fallos eléctricos, errores de redistribución o lesiones por presión durante las pruebas, manteniendo una tasa de incidentes igual a cero. Estos criterios no solo permitirán validar el correcto funcionamiento del sistema, sino también garantizarán que sea aceptado, confiable y autónomo para el uso diario por parte de adultos mayores con movilidad reducida.

Referencias bibliográficas

R. Fadil, B. Hoffmann, S. Lovelace, B. Farahani, S. Arzanpour, J. Loscheider, A. Aboonabi, and K. Tavakolian, "Design and evaluation of a dynamic air cushion for pressure ulcers prevention," *Journal of Tissue Viability*, vol. 31, no. 3, pp. 491–500, Aug. 2022. Available: <https://doi.org/10.1016/j.medengphy.2019.06.006>

R. Navickas, M. Stankus, V. Stankus, A. Laucevičius, A. Slušnienė, L. Ryliškytė, I. Laucevičienė, V. Kasiulevičius, and V. Laucevičius, "Sensor cell network for pressure, temperature and position detection on wheelchair users," *Int. J. Environ. Res. Public Health*, vol. 19, no. 4, p. 2195, Apr. 2022. [Online]. Available: <https://www.mdpi.com/1660-4601/19/4/2195>.

W. Carrigan, P. Nuthi, C. Pande, M. B. J. Wijesundara, C.-S. Chung, G. G. Grindle, J. D. Brown, B. Gebrosky, and R. A. Cooper, "Design and operation verification of an automated pressure mapping and modulating seat cushion for pressure ulcer prevention," *Medical Engineering & Physics*, vol. 69, pp. 17–27, Jul. 2019. Available: <https://doi.org/10.1016/j.jtv.2022.04.004>